

StreamForge

2-Week Implementation & Delivery Roadmap

A high-performance, Python-native distributed event streaming engine featuring RocksDB state persistence, dead-letter queue routing, and real-time observability.

Project Scope: Core Streaming, Fault Tolerance, Observability

Architecture: Kafka + RocksDB + Streamlit

Timeline: 14 Days (Week 1: Core Systems | Week 2: Integration & Resiliency)

Team Size: 5 Engineers

Executive Summary

StreamForge delivers enterprise-grade streaming capabilities in pure Python. Over a 14-day execution cycle, the team is building an end-to-end telemetry pipeline capable of processing high-throughput IoT sensor streams, validating schema integrity, maintaining zero-data-loss state across crashes using RocksDB, and visualizing cluster health in real time.

01. Team Architecture & Ownership Matrix

Module & File	Primary Owner	Core Responsibilities	Key Technical Output
<code>producer/producer.py</code>	MEMBER 1 (LEAD)	Simulates high-frequency IoT truck telemetry data; sends events to Kafka; implements stress-test mode.	Raw telemetry stream generation (<code>`truck-telemetry`</code>), configurable throughput control.
<code>workers/stream_worker.py</code>	VAISHNAVI (MEMBER 2)	Data quality filtering, JSON schema validation, boundary checks, dead-letter queue (DLQ) routing.	Clean data stream (<code>`clean-telemetry`</code>) & error routing (<code>`dead-letter-queue`</code>).
<code>state/state_manager.py</code>	PAVAN (MEMBER 3)	RocksDB local storage, manual offset commits, changelog replication, automated bootstrap state recovery.	RocksDB key-value store, changelog topic (<code>`state-changelog`</code>), recovery engine.
<code>dashboard/app.py</code>	SAI & AKSHAT (M4 & M5)	Real-time Streamlit dashboard, Plotly metric visualizers, cluster health tracking, DLQ logs.	Live Web UI monitoring metrics, state snapshot viewer, and system alert log.

02. Week 1: Core Engine & Pipeline Construction

The primary goal of Week 1 is establishing the end-to-end data pipeline locally across all team members' environments.

Day / Phase	Focus Area	Action Items & Tasks	Priority
Day 1 – 2	Environment & Producer Setup	- Finalize repository structure and Git workflow (`feature-*` branches). - Configure standard `docker-compose.yml` for Apache Kafka and Zookeeper/KRaft. - Implement `producer.py` to stream realistic truck JSON telemetry (ID, temp, speed, timestamp).	P1 - HIGH
Day 3 – 4	Worker & Data Validation Gate	- Vaishnavi develops `workers/stream_worker.py` using `kafka-python`. - Implement schema assertions, null checks, temperature boundary checks ($\\$-20^{\circ}\text{C}$ to $\\$150^{\circ}\text{C}$), and timestamp drift limits. - Implement Dead-Letter Queue (DLQ) for corrupt payloads.	P1 - HIGH
Day 5 – 6	RocksDB State Engine & Changelog	- Pavan integrates `rocksdict` embedded storage in `state/state_manager.py`. - Build rolling aggregator (events count, sum, max temp, average temp). - Write state updates into RocksDB and replicate to `state-changelog` topic. - Implement manual offset commits after successful state persistence.	P1 - HIGH
Day 7	Dashboard UI MVP	- Sai & Akshat launch initial Streamlit interface in `dashboard/app.py`. - Connect non-blocking Kafka consumers to `state-changelog` and `dead-letter-queue`. - Render live KPI metrics cards and fleet temperature bar charts via Plotly.	P2 - MEDIUM

03. Week 2: System Integration, Resiliency & Launch

Week 2 transforms individual working scripts into a hardened, fault-tolerant distributed system ready for live demonstration.

Day / Phase	Focus Area	Action Items & Tasks	Priority
Day 8	PR Merging & Integration	- Review and merge feature branches (`feature-worker`, `feature-state-manager`, `feature-dashboard`) into `main`. - Execute full end-to-end multi-process system test across all 4 terminal services.	P1 - HIGH
Day 9 – 10	Chaos & Crash Recovery Testing	Worker Failover: Force-kill worker process; verify Kafka consumer group rebalances without dropping records. State Recovery Test: Kill state manager, purge `./rocksdb_data`, restart script, and verify full automated restoration from `state-changelog`.	P1 - HIGH
Day 11	Stress Testing & Benchmarking	- Enhance `producer.py` with high-throughput mode (\$1,000 - 5,000 ext{ msg/sec}\$). - Measure consumer lag, RocksDB write latency, and UI refresh responsiveness under peak load.	P2 - MEDIUM
Day 12	Dashboard Polish & Health Monitoring	- Add system status indicators (Broker connection status, live events/sec counter). - Add RocksDB raw key-value table viewer and DLQ error inspection drawer.	P2 - MEDIUM
Day 13 – 14	Documentation & Demo Video	- Write comprehensive `README.md` with system architecture diagrams, single-command startup steps, and fault-tolerance design notes. - Record 5-minute live demonstration video featuring a real-time hard process kill and state restoration.	P3 - FINAL

04. Pipeline Architecture & Flow



Data Ingestion & Validation

- Producer:** Emits telemetry events to `truck-telemetry`.
- Worker (Vaishnavi):** Consumes raw telemetry and applies 5 validation rules (Schema, Type, Range, Nulls, Timestamp drift).
- Routing:** Valid events $\rightarrow$ `clean-telemetry`; Invalid events $\rightarrow$ `dead-letter-queue`.



State Store & Observability

- State Engine (Pavan):** Consumes `clean-telemetry`, calculates rolling stats, and commits to local RocksDB.
- Changelog:** Emits updates to `state-changelog` for backup & recovery.
- Dashboard (Sai & Akshat):** Monitors changelog and DLQ to display live analytics and system metrics.

05. Resiliency & Verification Testing Playbook

⚡ How to Demonstrate Zero-Data-Loss State Recovery

1. Launch all services: Kafka Docker, Producer, Worker, State Manager, and Dashboard.
2. Let the system process ~100 events so truck averages accumulate in RocksDB.
3. Simulate catastrophic disk failure: Force-kill `state_manager.py` (`Ctrl + C` or `kill -9`) and delete the `./rocksdb_data` folder.
4. Restart `python state/state_manager.py`. Observe the recovery sequence:
 Local RocksDB is empty. Rebuilding state from Kafka changelog...
5. Confirm on the Streamlit dashboard that all metric totals and averages are 100% restored with zero data loss.

StreamForge Engine Project Plan • Confidential & Internal Team Use • Built with Python, Kafka, RocksDB & Streamlit